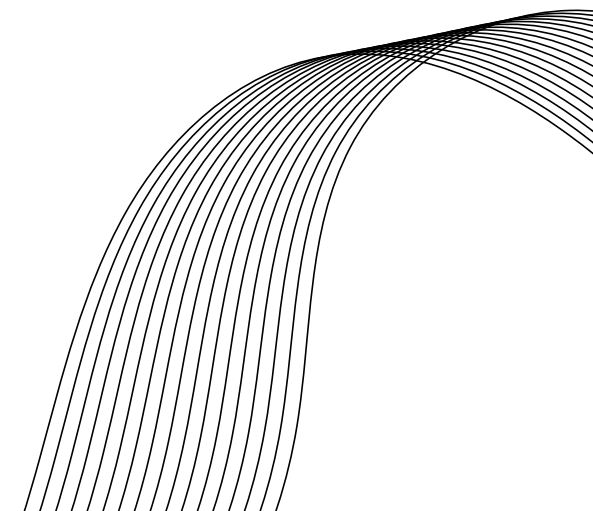




E-LIXO ZERO

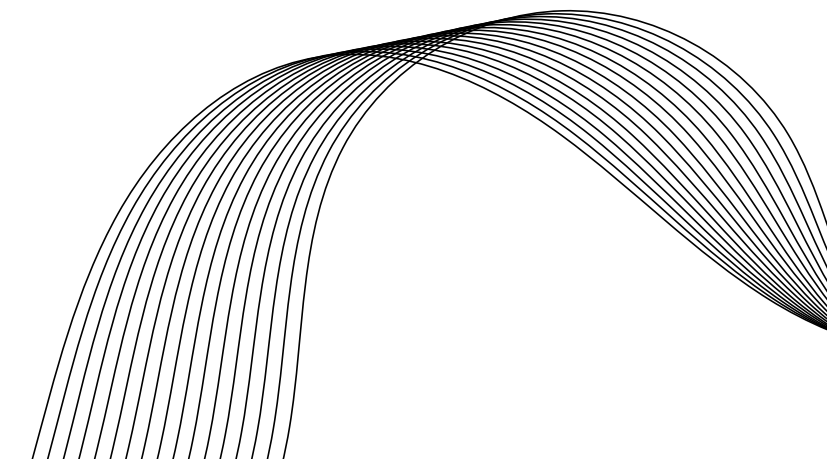
Conscientização ambiental sobre o descarte responsável
de equipamentos eletrônicos.



Sumário

- Problema e objetivos
- Escopo funcional
- Arquitetura geral
- Ports & Adapters
- Diagrama de pacotes
- Diagrama de classes
- Modelo de dados (MER e DDL)
- Segurança
- API Rest
- Tratamento de erros
- Regras de negócio
- Demonstração do projeto

- Decisões técnicas
- Dificuldades enfrentadas
- Conclusão





Problemas e objetivos

Problema

- Descarte irregular de residuos eletronicos na regioa
 - Cidadao nao sabe onde nem como descartar
 - Pontos de coleta sem divulgacao centralizada
- Ausencia de canal para agendar coleta residencial

Objetivo do software

- Centralizar informacoes sobre residuos e pontos de coleta
 - Permitir solicitacao e acompanhamento de coletas
 - Conectar cidadao, coletor e gestor publico
 - Notificar o usuario a cada mudanca de status

Requisitos nao funcionais atendidos

- Autenticacao stateless com JWT e senhas com hash BCrypt
- Autorizacao por perfil (CITIZEN, COLLECTOR, ADMIN) no back-end e no front-end
 - API REST com contrato JSON e codigos HTTP padronizados
 - Interface responsiva (desktop e celular)
- Baixo acoplamento: dominio isolado de framework e banco (Ports & Adapters)

Escopo funcional

CITIZEN(Cidadao)

- Cadastro e login
- Consulta de pontos de coleta (busca por cidade e por proximidade)
- Consulta de tipos de residuo
- Solicitacao de coleta
- Acompanhamento das coletas com status colorido
- Central de notificacoes
- Perfil do usuario

COLLECTOR(Coletor)

- Lista de coletas disponiveis e atribuidas a ele
- Busca e filtro por status
- Atualizacao de status: atribui a coleta automaticamente
- Nao altera coleta de outro coletor
- CONCLUIDA / CANCELADA sao estados finais (confirmacao em modal)

ADM

- Gestao de usuarios (ativar, desativar, trocar perfil, excluir)
- Gestao de pontos de coleta
- Gestao de tipos de residuo
- Gestao de todos os agendamentos
- Envio de notificacao individual ou broadcast

Arquitetura geral

Navegador

Angular 21
pages / services / guards

Controller

REST + DTO
validacao de contrato

Service (Porta)

regras de negocio adapter
injetado

DAO (Porta)

adapter Postgres
JDBC + PreparedStatement

- O **Controller** nunca conversa com o DAO: sempre passa pelo **Service**.
- O **Service** depende de **interfaces (ports)**, nunca da implementacao concreta do banco.
- Troca de banco = escrever um novo **adapter**, sem tocar em dominio nem em controller.
- Erros sobem como **excecoes tipadas** e sao convertidos em JSON pelo **GlobalExceptionHandler**.

Ports & Adapter

```
controller/      PickupsRestController,
                  UsersRestController, ...
dto/             LoginRequestDto, PickupRequestDto, ...
domain/          UserModel, PickupRequestModel, ...
ports_and_adapters/
  port/service/  PickupRequestService (interface)
  port/dao/      PickupRequestDao (interface)
  port/dao/crud/ CreateDao, ReadDao, UpdateDao, DeleteDao
  adapter/service/ PickupRequestServiceAdapter
  adapter/dao/    PickupRequestPostgresDaoAdapter
security/        JwtTokenService, JwtAuthenticationFilter
exceptions/      ApiException +
handler/GlobalExceptionHandler
configuration/    AppConfiguration, WebConfiguration
```

Por que essa divisao

- Dominio e regras nao conhecem Spring nem JDBC
- Interfaces CRUD segregadas (Create/Read/Update/Delete): cada DAO implementa so o que precisa
- Injecao das implementacoes em AppConfiguraton (@Bean)

Diagrama de pacotes

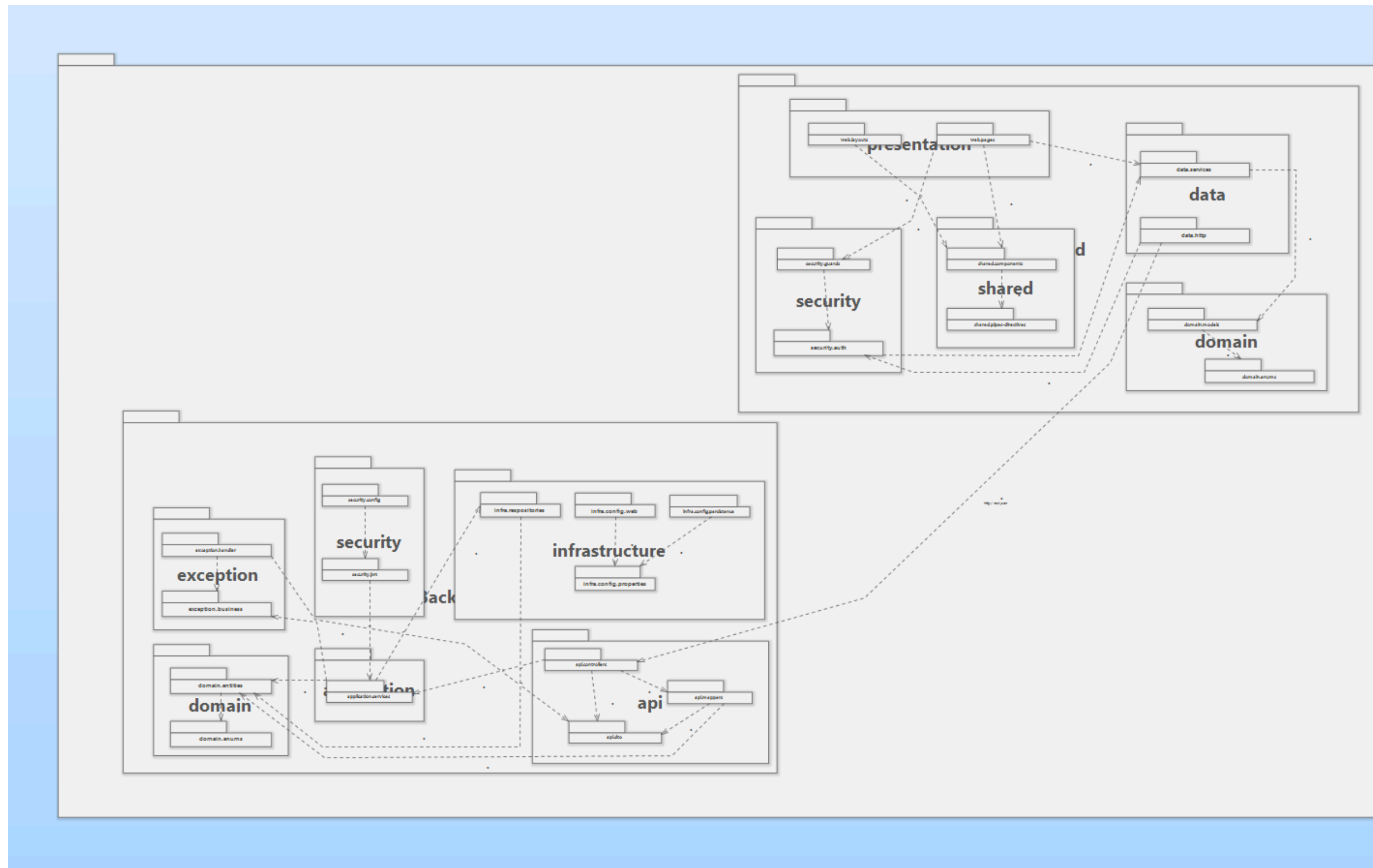
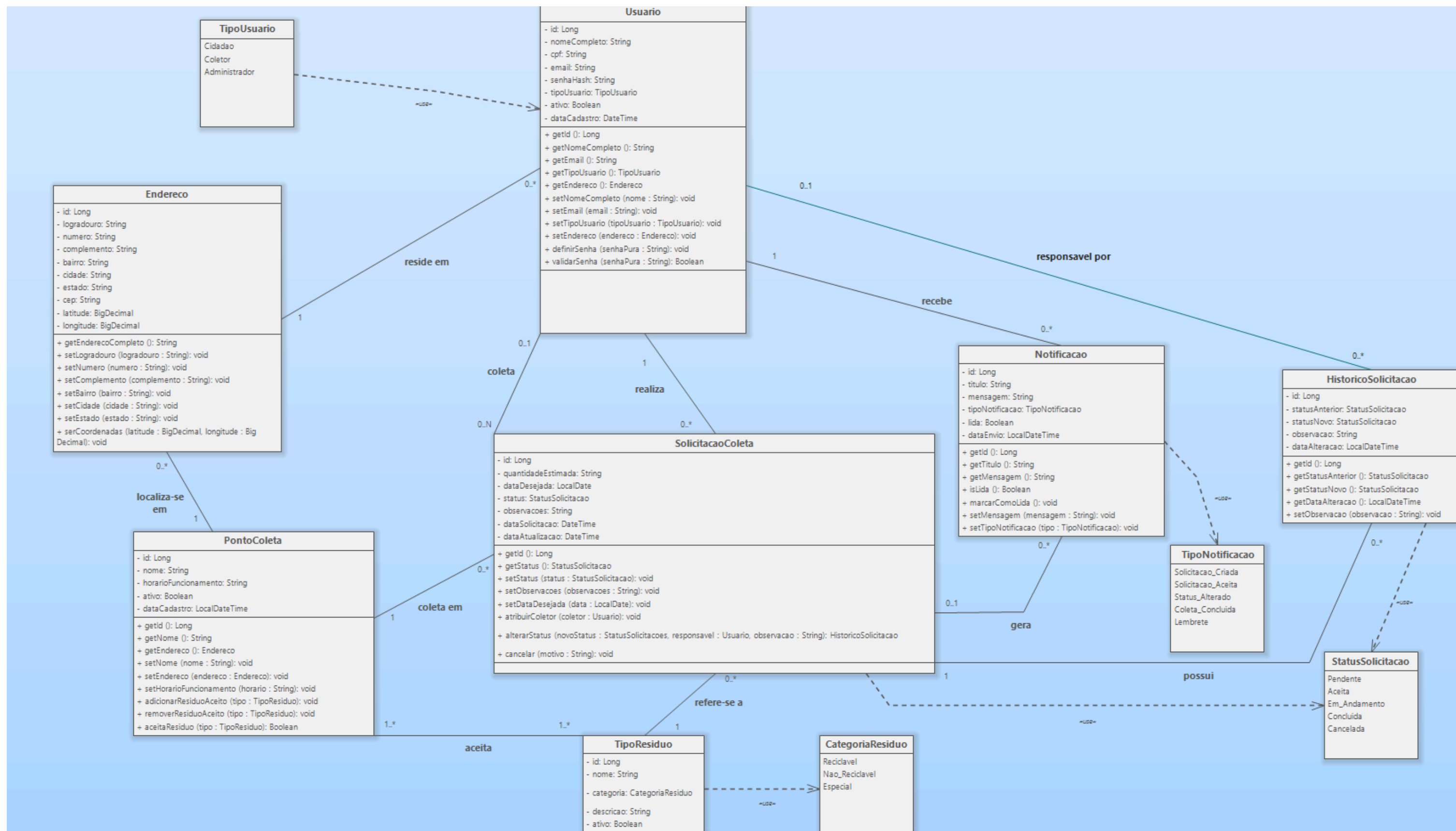
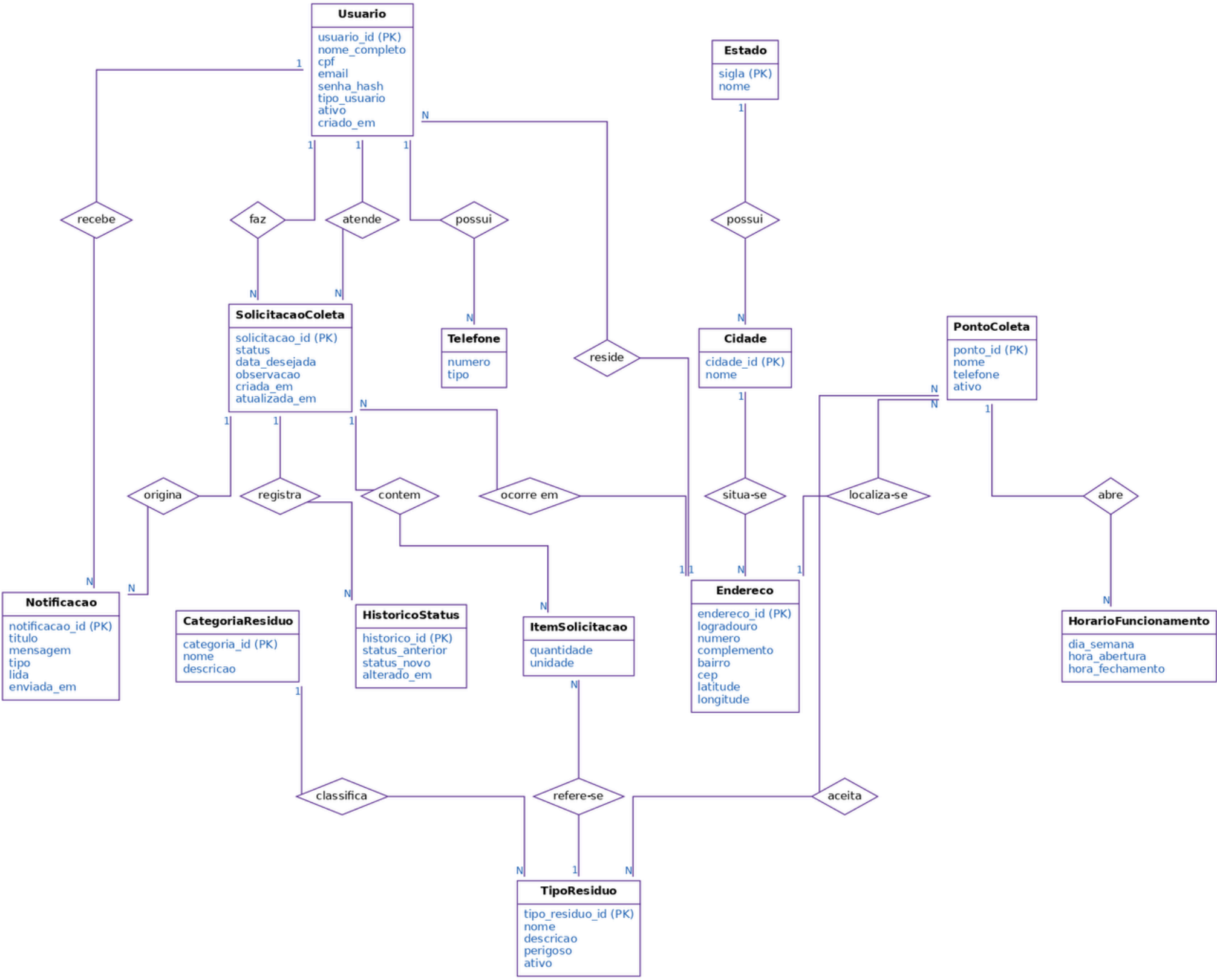


Diagrama de classes



Modelo Entidade e Relacionamento



DDL Banco de dados

```
CREATE TABLE users
(
  user_id SERIAL not null,
  full_name varchar(150) not null,
  cpf varchar(14) unique,
  email varchar(150) not null unique,
  phone varchar(20),
  street varchar(150),
  number varchar(20),
  neighborhood varchar(100),
  city varchar(100) default 'Santa Rita do Sapucaí',
  state varchar(2) default 'MG',
  password varchar(100) not null,
  user_type varchar(20) default 'CITIZEN',
  active boolean default true,
  created_at timestamp default current_timestamp,
  PRIMARY KEY (user_id)
);
```

```
CREATE TABLE waste_types
(
  waste_type_id SERIAL not null,
  name varchar(100) not null unique,
  category varchar(100),
  description text,
  active boolean default true,
  PRIMARY KEY (waste_type_id)
);

CREATE TABLE collection_points
(
  point_id SERIAL not null,
  name varchar(150) not null,
  street varchar(150) not null,
  number varchar(20),
  neighborhood varchar(100),
  city varchar(100) default 'Santa Rita do Sapucaí',
  state varchar(2) default 'MG',
  cep varchar(10),
  phone varchar(20),
  opening_hours varchar(100),
  accepted_waste_types text,
  latitude double precision,
  longitude double precision,
  active boolean default true,
  PRIMARY KEY (point_id)
);
```

DDL Banco de dados

```
CREATE TABLE pickup_requests
(
    pickup_id SERIAL not null,
    user_id int not null,
    waste_type_id int not null,
    collector_id int,
    street varchar(150) not null,
    number varchar(20),
    neighborhood varchar(100) not null,
    city varchar(100) default 'Santa Rita do Sapucaí',
    state varchar(2) default 'MG',
    estimated_quantity varchar(100),
    desired_date date,
    status varchar(30) default 'PENDING',
    notes text,
    requested_at timestamp default current_timestamp,
    updated_at timestamp,
    PRIMARY KEY (pickup_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (waste_type_id) REFERENCES waste_types(waste_type_id),
    FOREIGN KEY (collector_id) REFERENCES users(user_id)
);
```

```
CREATE TABLE notifications
(
    notification_id SERIAL not null,
    user_id int not null,
    title varchar(100) not null,
    message text not null,
    notification_type varchar(30) default 'INFO',
    is_read boolean default false,
    sent_at timestamp default current_timestamp,
    PRIMARY KEY (notification_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

Seguranca

Autenticacao stateless com JWT e autorizacao por perfil

Back-end

- Senha nunca trafega nem e armazenada em texto puro: BCrypt (spring-security-crypto)
- Login valida credenciais e emite JWT
- JwtAuthenticationFilter (OncePerRequestFilter): valida o token, resolve o usuario e bloqueia inativos
- Rotas publicas explicitas (login, cadastro, GETs publicos)

Front-end

- auth.interceptor.ts injeta o header Authorization
- auth-guard: exige usuario autenticado
- admin-guard e collector-guard: exigem o perfil correto
- Token e perfil guardados no storage do navegador
- Guard e apenas UX: a autorizacao real e no servidor

API Rest

	POST	/api/users	cadastro (gera notificacao de boas-vindas)	201 / 400
POST	/api/users/login	autenticacao, devolve JWT	200 / 401	
GET	/api/collection-points	lista pontos de coleta	200	
GET	/api/collection-points/nearby	?lat=..&lng=.. ordenado por distancia (Haversine)	200	
GET	/api/waste-types	catalogo publico de residuos	200	
GET	/api/pickups	coletas do usuario autenticado	200 / 401	
POST	/api/pickups	solicita coleta	201 / 400	
GET	/api/pickups/collector	fila do coletor (disponiveis + atribuidas)	200 / 403	
PUT	/api/pickups/{id}/status	atualiza status, atribui coletor e notifica	200 / 403 / 404	
GET	/api/pickups/all	todas as coletas (ADMIN)	200 / 403	
POST	/api/notifications/broadcast	notificacao para todos os ativos (ADMIN)	201 / 403	

- DTO de entrada e saida separados dos modelos de dominio (PickupRequestDto / PickupResponseDto).
- 201 Created com header Location no cadastro de recursos (ServletUriComponentsBuilder).
- Mesma rota PUT /pickups/{id}/status atende coletor e admin, com autorizacao distinta por perfil.

Tratamento de erros

```
        ApiException (base, carrega statusCode)
|- BadRequestException          -> 400
|- UnauthorizedException        -> 401 / 403
|- ResourceNotFoundException    -> 404

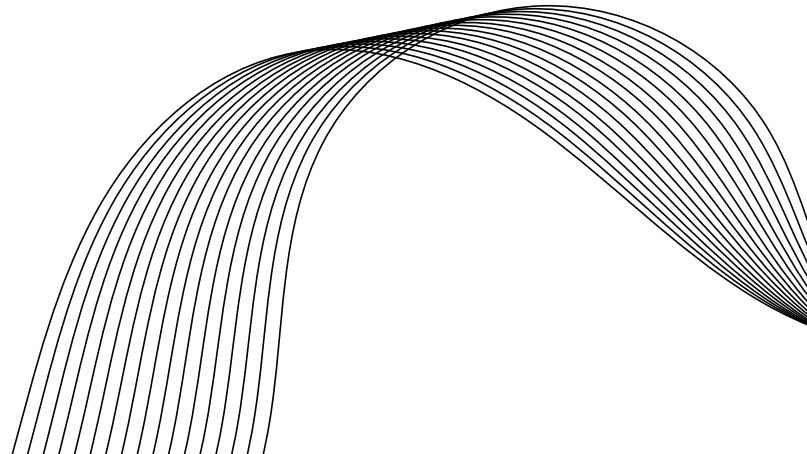
@RestControllerAdvice
class GlobalExceptionHandler {
    @ExceptionHandler(ApiException.class)
    -> { timestamp, status, error }
    @ExceptionHandler(Exception.class)
    -> 500 Internal Server Error
}
```

Exemplo de resposta

```
{
  "timestamp": "2026-09-18T20:14:03",
  "status": 404,
  "error": "Coleta nao encontrada"
}
```



Regras de negócio implementadas

- **Atribuicao automatica do coletor:** ao atualizar o status, a coleta é vinculada ao coletor autenticado; coleta de outro coletor não pode ser alterada.
 - **Estados finais:** CONCLUIDA e CANCELADA não admitem nova transição (validado no back-end e confirmado em modal no front).
 - **Notificacao automatica:** cadastro gera notificação de boas-vindas; toda mudança de status notifica o solicitante.
 - **Broadcast administrativo:** envio de notificação para todos os usuários ativos em uma única operação.
 - **Pontos por proximidade:** cálculo de distância pela fórmula de Haversine (raio 6371 km) e ordenação crescente, com arredondamento para 2 casas.
 - **Revogacao imediata de acesso:** usuário desativado ou excluído é barrado no próximo request, mesmo com token válido em mãos.
 - **Busca de pontos por cidade, bairro ou nome, em uma única consulta.**
- 

DESMONSTRAÇÃO DO PROJETO



[Início](#)[Sobre](#)[Como funciona](#)[Pontos de Coleta](#)

[Entrar](#)[Cadastre-se](#)

Descarte correto, planeta melhor.

O E-Lixo Zero conecta você aos pontos de coleta mais próximos e facilita o descarte consciente de resíduos eletrônicos.

[Solicitar Coleta](#)[Ver Pontos de Coleta](#)





Descarte Consciente
Ajude o meio ambiente



Coletas Agendadas
Agende em segundos



Pontos de Coleta
Encontre o mais próximo



Notificações
Acompanhe suas coletas



DTO separado do modelo de dominio

- + Contrato da API estavel e independente do banco;
nao expoe hash de senha
- Exige conversao explicita entre camadas

Ports & Adapters

- + Troca de tecnologia de persistencia sem alterar regra de negocio; testavel com mocks
- Mais interfaces e arquivos para um projeto pequeno



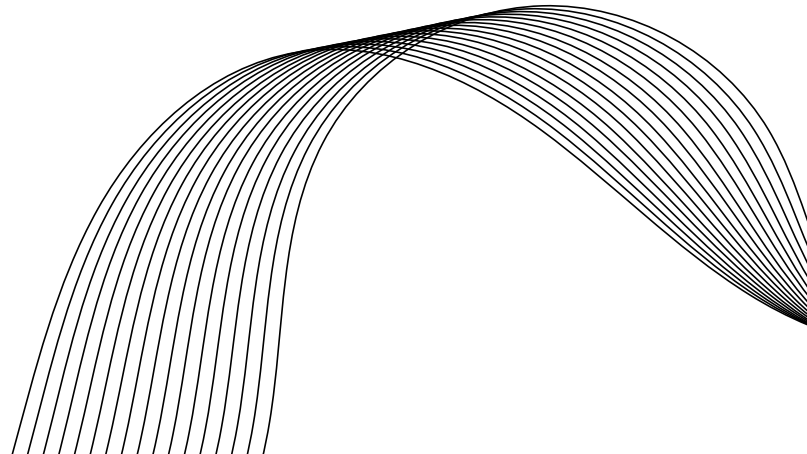
Dificuldades enfrentadas

- **Sessao de usuario desativado continuar valida** - o filtro passou a reconsultar o usuario a cada request, e nao apenas validar a assinatura do token.
- **Transicoes de status inconsistentes** - centralizacao das regras no service, com estados finais bloqueados.
- **Duplicacao de endereco nas entidades** - identificada na revisao dos diagramas; modelada como estrutura reutilizavel.
- **Acoplamento entre controller e banco** - eliminado com as portas (interfaces) e adapters.
- **Retornos genericos da API** - substituidos por excecoes tipadas e codigos HTTP corretos.



Conclusão

O E-Lixo Zero centraliza informações, incentiva o descarte consciente e aproxima a população dos serviços de coleta, contribuindo para a sustentabilidade ambiental.



OBRIGADO

